

Exercícios de autorrevisão

3.1 Preencha os espaços nas seguintes sentenças:

- O procedimento para resolver um problema em termos das ações a serem executadas e da ordem em que essas ações devem ser executadas é chamado de _____.
- Especificar a ordem de execução das instruções pelo computador é chamado de _____.
- Todos os programas podem ser escritos nos termos de três tipos de estruturas de controle: _____, _____ e _____.
- A estrutura de seleção _____ é usada para executar uma ação quando uma condição é verdadeira e outra ação quando essa condição é falsa.
- Várias instruções agrupadas com chaves ({ e }) são chamadas de _____.
- A estrutura de repetição _____ especifica que uma instrução ou grupo de instruções deve ser executada repetidamente enquanto alguma condição permanecer verdadeira.
- A repetição de um conjunto de instruções por um número específico de vezes é chamada de repetição _____.
- Quando não se sabe com antecedência quantas vezes um conjunto de instruções será repetido, um valor de _____ pode ser usado para encerrar a repetição.

3.2 Escreva quatro instruções diferentes em C; cada uma deve somar 1 à variável inteira x.

3.3 Escreva uma única instrução em C que possibilite realizar as seguintes tarefas:

- atribuir a soma de x e y a z e incrementar o valor de x em 1 após o cálculo.
- multiplicar a variável produto por 2 usando o operador `*`.
- multiplicar a variável produto por 2 usando os operadores `=` e `*`.
- testar se o valor da variável contador é maior do que 10. Se for, imprimir 'Contador é maior do que 10'.
- decrementar a variável x em 1, depois subtraí-la da variável total.
- somar a variável x à variável total, depois decrementar x por 1.
- calcular o resto após q ser dividido por divisor e atribuir o resultado a q; escreva essa instrução de duas maneiras diferentes.
- imprimir o valor 123.4567 com 2 dígitos de precisão. Que valor é impresso?
- imprimir o valor de ponto flutuante 3.14159 com três dígitos à direita do ponto decimal. Que valor é impresso?

3.4 Escreva uma instrução em C que possibilite realizar as seguintes tarefas:

- declarar variáveis soma e x para que sejam do tipo `int`.
- inicializar variável x em 1.
- inicializar variável soma em 0.
- somar variável x à variável soma e atribuir o resultado à variável soma.
- imprimir 'A soma é:' seguido pelo valor da variável soma.

3.5 Combine as instruções que você escreveu no Exercício 3.4 em um programa que calcule a soma dos inteiros de 1 a 10. Use a estrutura `while` para realizar um loop pelas instruções de cálculo e incremento. O loop deverá terminar quando o valor de x chegar a 11.

3.6 Determine os valores das variáveis produto e x depois que o cálculo a seguir for realizado. Suponha que produto e x tenham o valor 5 quando a instrução começar a ser executada.

```
produto *= x++;
```

3.7 Escreva instruções em C que:

- leiam a variável inteira x com `scanf`.
- leiam a variável inteira y com `scanf`.
- inicializem a variável inteira i em 1.
- inicializem a variável inteira potência em 1.
- multipliquem a variável potência por x e atribua o resultado à potência.
- incrementem a variável i em 1.
- testem i para verificar se ele é maior ou igual a y na condição de uma estrutura `while`.
- exibam a variável inteira potência com `printf`.

3.8 Escreva um programa em C que use as instruções do Exercício 3.7 para calcular x elevado à potência y. O programa deverá ter uma estrutura de controle de repetição `while`.

3.9 Identifique e corrija os erros em cada um dos itens:

- ```
while (c <= 5) {
 produto *= c;
 ++c;
}
```
- ```
scanf( "%.4f", &valor );
```
- ```
if (sexo == 1)
 printf("Mulher\n");
se não
 printf("Homem\n");
```

### 3.10 O que está errado na estrutura de repetição `while` a seguir (considere z com valor 100), que supõe-se calcular a soma dos inteiros de 100 para 1?

```
while (z >= 0)
 soma + = z;
```

## Respostas dos exercícios de autorrevisão

**3.1** a) Algoritmo. b) Controle do programa. c) Sequência, seleção, repetição. d) if...else. e) Instrução composta. f) while. g) Controlada por contador. h) Sentinela.

**3.2** `x = x + 1;`  
`x += 1;`  
`++x;`  
`x++;`

**3.3** a) `z = x++ + y;`  
 b) `produto *= 2;`  
 c) `produto = produto * 2;`  
 d) `if ( contador > 10 )`  
     `printf( "Contador é maior que 10.\n" );`  
 e) `total -= --x;`  
 f) `total += x--;`  
 g) `q %= divisor;`  
     `q = q % divisor;`  
 h) `printf( "%.2f", 123.4567 );`  
     123.46 é exibido.  
 i) `printf( "%.3f\n", 3.14159 );`  
     3.142 é exibido.

**3.4** a) `int soma, x;`  
 b) `x = 1;`  
 c) `soma = 0;`  
 d) `soma += x;` ou `soma = soma + x;`  
 e) `printf( "A soma é: %d\n", soma );`

**3.5** Veja na listagem de programa a seguir.

```
1 /* Calcula a soma dos inteiros de 1 a 10 */
2 #include <stdio.h>
3
4 int main(void)
5 {
6 int soma, x; /* declara variáveis soma e x */
7
8 x = 1; /* inicializa x */
9 soma = 0; /* inicializa soma */
10
11 while (x <= 10) { /* loop while x é menor
12 soma += x; /* soma x à variável soma */
13 ++x; /* incrementa x */
14 } /* fim do while */
15
16 printf("A soma é: %d\n", soma); /* exibe a soma */
17 return 0;
18 } /* fim da função main */
```

**3.6** `produto = 25, x = 6;`

**3.7** a) `scanf( "%d", &x );`  
 b) `scanf( "%d", &y );`  
 c) `i = 1;`  
 d) `potência = 1;`  
 e) `potência *= x;`  
 f) `i++;`  
 g) `if ( i <= y )`  
 h) `printf( "%d", potência );`

**3.8** Veja a seguir.

```
1 /* eleva x à potência y */
2 #include <stdio.h>
3
4 int main(void)
5 {
6 int x, y, i, potência; /* declara variáveis */
7
8 i = 1; /* inicializa i */
9 potência = 1; /* inicializa potência */
10 scanf("%d", &x); /* lê do usuário o valor para x */
11 scanf("%d", &y); /* lê do usuário o valor para y */
12
13 while (i <= y) { /* loop while i é menor ou igual a y */
14 potência *= x; /* multiplica potência por x */
15 ++i; /* incrementa i */
16 } /* fim do while */
17
18 printf("%d", potência); /* exibe potência */
19 return 0;
20 } /* fim da função main */
```

**3.9** a) Erro: Falta a chave à direita do while.

Correção: Adicione a chave à direita depois da estrutura `++c;`.

b) Erro: Precisão usada na especificação de conversão `scanf`.

Correção: Retire `.4` da especificação de conversão.

c) Erro: Ponto e vírgula depois de else em uma estrutura if...else resulta em um erro lógico. O segundo `printf` continuará a ser executado.

Correção: Retire o ponto e vírgula que parece depois de else.

**3.10** O valor da variável `z` nunca muda na estrutura while. Portanto, cria-se um loop infinito. Para impedir que isso aconteça, `z` precisa ser decrementado, de modo que seu valor chegue a 0.

## Exercícios

**3.11** Identifique e corrija os erros dos seguintes trechos e códigos: [Nota: pode haver mais de um erro em cada um.]

- a) `if ( idade >= 65 );`  
     `printf( "Idade é maior ou igual a 65\n" );`  
     **else**  
     `printf( "Idade é menor que 65\n" );`
- b) `int x = 1, total;`  
     **while** ( x <= 10 ) {  
         `total += x;`  
         `++x;`  
     }
- c) `while ( x <= 100 )`  
     `total += x;`  
     `++x;`
- d) `while ( y > 0 ) {`  
     `printf( "%d\n", y );`  
     `++y;`  
     }

**3.12** Preencha os espaços em cada sentença:

- a) A solução para qualquer problema envolve realizar uma série de ações em um(a) \_\_\_\_\_ específico(a).
- b) Um sinônimo para procedimento é \_\_\_\_\_.
- c) Uma variável que acumula a soma de vários números é um(a) \_\_\_\_\_.
- d) A definição de certas variáveis com valores específicos no início de um programa é chamada de \_\_\_\_\_.
- e) Um valor especial usado para indicar o 'fim da entrada de dados' é chamado de valor \_\_\_\_\_, \_\_\_\_\_ ou \_\_\_\_\_.
- f) Um(a) \_\_\_\_\_ é uma representação gráfica de um algoritmo.
- g) Em um fluxograma, a ordem em que as etapas devem ser realizadas é indicada por símbolos \_\_\_\_\_.
- h) O símbolo de finalização indica o(a) \_\_\_\_\_ e \_\_\_\_\_ de cada algoritmo.
- i) Retângulos correspondem a cálculos que normalmente são realizados por instruções e operações de entrada/saída que normalmente são realizadas por chamadas às funções \_\_\_\_\_ e \_\_\_\_\_ da biblioteca-padrão.
- j) O item escrito dentro de um símbolo de decisão é chamado de \_\_\_\_\_.

**3.13** O que o programa a seguir imprime?

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int x = 1, total = 0, y;
6
7 while (x <= 10) {
8 y = x * x;
9 printf("%d\n", y);
10 total += y;
11 ++x;
12 } /* fim do while */
13
14 printf("Total is %d\n", total);
15 return 0;
16 } /* fim do main */
```

**3.14** Escreva uma única instrução em pseudocódigo que indique cada um dos seguintes comandos:

- a) Apresente a mensagem 'Digite dois números'.
- b) Atribua a soma das variáveis x, y e z à variável p.
- c) A seguinte condição deve ser testada em uma estrutura de seleção if...else: o valor atual da variável m é maior que o dobro do valor atual da variável v.
- d) Obtenha valores para as variáveis s, r e t pelo teclado.

**3.15** Formule um algoritmo em pseudocódigo para cada um dos seguintes comandos:

- a) Obter dois números a partir do teclado, calcular sua soma e exibir o resultado.
- b) Obter dois números a partir do teclado, determinar e exibir qual deles (se houver algum) é o maior dos dois números.
- c) Obter uma série de números positivos a partir do teclado, determinar e exibir sua soma. Suponha que o usuário digite um valor de sentinela -1 para indicar o 'fim da entrada de dados'.

**3.16** Indique quais das sentenças seguintes são *verdadeiras* e quais são *falsas*. Caso haja uma afirmação *falsa*, explique.

- a) A experiência tem mostrado que a parte mais difícil da solução de um programa em um computador é a produção de um programa em C funcional.
- b) Um valor de sentinela deve ser um valor que não possa ser confundido com um valor de dados legítimo.
- c) Linhas de fluxo indicam as ações a serem realizadas.
- d) Condições escritas dentro de símbolos de decisão sempre contêm operadores aritméticos (ou seja, +, -, \*, / e %).

- e) Nos refinamentos sucessivos top-down, cada refinamento é uma representação completa do algoritmo.

**Para os exercícios 3.17 a 3.21, realize cada uma destas etapas:**

1. Leia o enunciado do problema.
2. Formule o algoritmo usando pseudocódigo e refinamentos sucessivos top-down.
3. Escreva um programa em C.
4. Teste, depure e execute o programa em C.

**3.17 Consumo de gasolina.** Os motoristas se preocupam com o consumo de combustível de seus automóveis. Um motorista manteve um registro do número de abastecimentos que fez, registrando também o número de quilômetros rodados e de litros obtidos a cada abastecimento. Desenvolva um programa que peça o número de quilômetros dirigidos e a quantidade de litros obtidos a cada abastecimento. O programa deverá calcular e exibir a quantidade de quilômetros rodados por litros usados. Depois de processar toda a informação, o programa deverá calcular e exibir o total combinado de quilômetros por litro para todos os abastecimentos. Veja um exemplo do diálogo de entrada/saída:

```
Informe quantos litros abasteceu (-1 para terminar): 25,6
Informe quantos km dirigiu: 287
O consumo atual é de 11,210937 km/l

Informe quantos litros abasteceu (-1 para terminar): 20,6
Informe quantos km dirigiu: 200
O consumo atual é de 9,708738 km/l

Informe quantos litros abasteceu (-1 para terminar): 10
Informe quantos km dirigiu: 120
O consumo atual é de 12,000000 km/l

Informe quantos litros abasteceu (-1 para terminar): -1
O consumo geral foi de 10,800712 km/l
```

**3.18 Calculadora de limite de crédito.** Desenvolva um programa em C que determine se um cliente de uma loja de departamentos excedeu seu limite de crédito. Os seguintes fatos estão disponíveis para cada cliente:

- a) Número de conta.
- b) Saldo no início do mês.
- c) Total de todos os encargos desse cliente nesse mês.
- d) Total de todos os créditos aplicados à conta desse cliente nesse mês.
- e) Limite de crédito autorizado.

O programa deverá ler cada um desses fatos, calcular o novo saldo ( $= \text{saldo inicial} + \text{encargos} - \text{créditos}$ ) e determinar se o novo saldo é superior ao limite de crédito do cliente. Para os clientes cujo limite de crédito foi ultrapassado, o programa deverá exibir o número de conta do cliente, o limite de crédito, o novo saldo e a mensagem 'Limite de crédito ultrapassado'. Veja um exemplo do diálogo de entrada/saída:

```
Informe o número da conta (-1 para terminar): 100
Informe o saldo inicial: 5394,78
Informe o total de encargos: 1000,00
Informe o total de créditos: 500,00
Informe o limite de crédito: 5500,00
Conta: 100
Limite de crédito: 5500,00
Saldo: 5894,78
Limite de crédito ultrapassado.
```

```
Informe número da conta (-1 para terminar): 200
Informe o saldo inicial: 1000,00
Informe o total de encargos: 123,45
Informe o total de créditos: 321,00
Informe o limite de crédito: 1500,00
```

```
Informe número da conta (-1 para terminar): 300
Informe o saldo inicial: 500,00
Informe o total de encargos: 274,73
Informe o total de créditos: 100,00
Informe o limite de crédito: 800,00
```

```
Informe número da conta (-1 para terminar): -1
```

**3.19 Calculadora de comissão de vendas.** Uma grande companhia química paga seus vendedores por comissão. Os vendedores recebem R\$ 200,00 por semana, mais 9 por cento de suas vendas brutas nessa semana. Por exemplo, um vendedor que comercialize R\$ 5.000,00 em produtos químicos em uma semana receberá R\$ 200,00 e mais 9 por cento de R\$ 5.000,00, ou seja, receberá um total de R\$ 650,00. Desenvolva um programa que peça os valores brutos de cada vendedor na semana que passou, calcule e apresente o valor que esse vendedor deverá receber. Processe os valores referentes a um vendedor por vez. Veja um exemplo do diálogo de entrada/saída:

```
Informe a venda em reais (-1 para terminar): 5.000,00
O pagamento é de: R$650,00
```

```
Informe a venda em reais (-1 para terminar): 1.234,56
O pagamento é de: R$311,11
```

```
Informe a venda em reais (-1 para terminar): 1.088,89
O pagamento é de: R$298,00
```

```
Informe a venda em reais (-1 para terminar): -1
```

**3.20 Calculadora de juros.** Os juros simples sobre um empréstimo são calculados a partir da fórmula

$$\text{juros} = \text{principal} * \text{taxa} * \text{dias} / 365;$$

Essa fórmula considera que taxa seja a taxa de juros anual e, portanto, inclui a divisão por 365 (dias). Desenvolva um programa que aceite principal, taxa e dias para vários empréstimos, calcule e apresente os juros simples para cada empréstimo, usando a fórmula apresentada. Veja um exemplo do diálogo de entrada/saída:

```
Informe o valor principal do empréstimo
(-1 para terminar): 1.000,00
Informe a taxa de juros: 0,1
Informe o prazo do empréstimo em dias: 365
O valor dos juros é de R$100,00

Informe o valor principal do empréstimo
(-1 para terminar): 1.000,00
Informe a taxa de juros: 0,08375
Informe o prazo do empréstimo em dias: 224
O valor dos juros é de R$51,40

Informe o valor principal do empréstimo
(-1 para terminar): 10.000,00
Informe a taxa de juros: 0,09
Informe o prazo do empréstimo em dias: 1460
O valor dos juros é de R$3600,00

Informe valor principal do empréstimo
(-1 para terminar): -1
```

**3.21 Calculadora de salário.** Desenvolva um programa que determine o salário semanal bruto de vários funcionários. A empresa paga 'uma hora normal' para cada funcionário pelas primeiras 40 horas trabalhadas, 'uma hora normal e meia' por hora trabalhada a partir de 40 horas. Você recebe uma lista de funcionários da empresa, o número de horas que cada funcionário trabalhou na semana anterior e o valor ganho por hora de cada funcionário. Seu programa deverá ler essas informações para cada funcionário e determinar e exibir o salário que cada um deverá receber. Aqui está um exemplo do diálogo de entrada/saída:

```
Digite # de horas trabalhadas (-1 para terminar): 39
Digite o salário por hora do funcionário (R$00,00): 10,00
Salário é de R$390,00

Digite # de horas trabalhadas (-1 para terminar): 40
Digite o salário por hora do funcionário (R$00,00): 10,00
Salário é de R$400,00

Digite # de horas trabalhadas (-1 para terminar): 41
Digite o salário por hora do funcionário (R$00,00): 10,00
Salário é de R$415,00

Digite # de horas trabalhadas (-1 para terminar): -1
```

**3.22 Pré-incrementando versus pós-incrementando.**

Escreva um programa que demonstre a diferença entre pré-decrementar e pós-decrementar usando o operador de decremento —.

**3.23 Imprimindo números por um loop.** Escreva um programa que utilize o looping para imprimir os números de 1 a 10 lado a lado na mesma linha, com três espaços entre os números.

**3.24 Ache o número maior.** O processo de achar o número maior (ou seja, o máximo de um grupo de números) é usado com frequência nas aplicações de computador. Por exemplo, um programa que determina o vencedor de uma disputa de vendas lerá o número de unidades vendidas por vendedor. O vendedor que tiver vendido mais unidades vence a disputa. Escreva um programa em pseudocódigo e depois um programa que leia uma série de 10 números, determine e imprima o maior dos números. [Dica: seu programa deverá usar três variáveis da seguinte forma]:

contador: Um contador para contar até 10 (ou seja, registrar quantos números foram informados e determinar quando os 10 números foram processados)

número: O número da entrada atual no programa

maior: O maior número achado até o momento

**3.25 Saída tabular.** Escreva um programa que use o looping para imprimir a tabela de valores a seguir. Use a sequência de escape de tabulação, \t, na instrução printf para separar as colunas com tabulações.

| N  | 10*N | 100*N | 1000*N |
|----|------|-------|--------|
| 1  | 10   | 100   | 1000   |
| 2  | 20   | 200   | 2000   |
| 3  | 30   | 300   | 3000   |
| 4  | 40   | 400   | 4000   |
| 5  | 50   | 500   | 5000   |
| 6  | 60   | 600   | 6000   |
| 7  | 70   | 700   | 7000   |
| 8  | 80   | 800   | 8000   |
| 9  | 90   | 900   | 9000   |
| 10 | 100  | 1000  | 10000  |

**3.26 Saída tabular.** Escreva um programa que use o looping para produzir a seguinte tabela de valores:

| A  | A+2 | A+4 | A+6 |
|----|-----|-----|-----|
| 3  | 5   | 7   | 9   |
| 6  | 8   | 10  | 12  |
| 9  | 11  | 13  | 15  |
| 12 | 14  | 16  | 18  |
| 15 | 17  | 19  | 21  |

**3.27 Ache os dois maiores números.** Usando uma técnica semelhante à do Exercício 3.24, ache os dois maiores valores dos 10 números. [Nota: você poderá informar cada número apenas uma vez.]

**3.28 Valide a entrada do usuário.** Modifique o programa da Figura 3.10 para validar suas entradas. Em qualquer entrada, se o valor inserido for diferente de 1 ou 2, continue o looping até que o usuário informe um valor correto.

**3.29** O que o programa a seguir exibe?

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int contador = 1; /* inicializa contador */
6
7 while (contador <= 10) { /* loop 10 vezes */
8
9 /* exibe linha de texto */
10 printf("%s\n", contador % 2 ? "*****" :
11 "+++++++");
12 contador++; /* incrementa contador */
13 } /* fim do while */
14
15 return 0; /* indica que o programa terminou com sucesso */
16 } /* fim da função main */
```

**3.30** O que o programa a seguir exibe?

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int linha = 10; /* inicializa linha */
6 int coluna; /* declara coluna */
7
8 while (linha >= 1) { /* loop até linha < 1 */
9 coluna = 1; /* define coluna como 1 quando a iteração começa */
10
11 while (coluna <= 10) { /* loop 10 vezes */
12 printf("%s", linha % 2 ? "<": ">"); /* saída */
13 coluna++; /* incrementa coluna */
14 } /* fim do while interno */
15
16 linha--; /* decrementa linha */
17 printf("\n"); /* inicia nova linha de saída */
18 } /* fim do while externo */
19
20 return 0; /* indica que o programa foi concluído com sucesso */
21 } /* fim da função main */
```

**3.31 Problema do else pendurado.** Determine a saída para cada um dos seguintes códigos quando x for 9 e y for 11, e quando x for 11 e y for 9. O compilador ignora o recuo em um programa em C. Além disso, o compilador sempre associa um else com o if anterior, a menos que seja informado de que deve proceder de outra forma pela colocação das chaves { }. Visto que, à primeira vista, você pode não saber ao certo a que if um else corresponde, isso é conhecido como o problema do 'else pendurado'. Eliminamos o recuo do código a seguir para tornar o problema mais desafiador. [Dica: aplique as convenções de recuo que você aprendeu.]

```
a) if (x < 10)
 if (y > 10)
 printf("*****\n");
 else
 printf("#####\n");
 printf("$$$$$\n");

b) if (x < 10) {
 if (y > 10)
 printf("*****\n");
 }
 else {
 printf("#####\n");
 printf("$$$$$\n");
 }
```

**3.32 Outro problema do else pendurado.** Modifique o seguinte código para produzir a saída mostrada. Use as técnicas de recuo apropriadas. Você pode não precisar fazer mudança alguma, além de inserir as chaves. O compilador ignora os recuos em um programa. Eliminamos o recuo do código a seguir para tornar o problema mais desafiador. [Nota: é possível que nenhuma modificação seja necessária.]

```
if (y == 8)
if (x == 5)
 printf("@@@@\n");
else
 printf("#####\n");
 printf("$$$$$\n");
 printf("#####\n");
```

a) Supondo que x = 5 e y = 8, a seguinte saída será produzida.

```
@@@@@
$$$$$
#####
```

b) Supondo que x = 5 e y = 8, a seguinte saída será produzida.

```
@@@@@
```

- c) Supondo que  $x = 5$  e  $y = 8$ , a seguinte saída será produzida.

```
@@@@@
&&&&&
```

- d) Supondo que  $x = 5$  e  $y = 7$ , a seguinte saída será produzida. [Nota: as três últimas instruções printf fazem parte de uma instrução composta.]

```
#####
$$$$$
&&&&&
```

- 3.33 Quadrado de asteriscos.** Escreva um programa que leia o lado de um quadrado e depois exiba esse quadrado a partir de asteriscos. Seu programa deverá funcionar para quadrados de todos os tamanhos de lado entre 1 e 20. Por exemplo, se o programa ler um tamanho 4, ele deverá exibir.

```



```

- 3.34 Quadrado de asteriscos vazio.** Modifique o programa que você escreveu no Exercício 3.33 para que ele exiba um quadrado vazio. Por exemplo, se seu programa ler um tamanho 5, ele deverá exibir

```

* *
* *
* *
* *

```

- 3.35 Testador de palíndromo.** Um palíndromo é um número, ou uma frase textual, que pode ser lido da mesma forma da esquerda para a direita e vice-versa. Por exemplo, cada um dos seguintes inteiros de cinco dígitos é um palíndromo: 12321, 55555, 45554 e 11611. Escreva um programa que leia um inteiro de cinco dígitos e determine se ele é ou não um palíndromo. [Dica: use os operadores de divisão e módulo para separar o número em seus dígitos individuais.]

- 3.36 Imprimindo o equivalente decimal de um número binário.** Leia um inteiro contendo apenas 0s e 1s (ou seja, um inteiro 'binário') e imprima seu equivalente decimal. [Dica: use os operadores de módulo e divisão para apanhar os dígitos do número 'binário' um de cada vez, da direita para a esquerda. Assim como no sistema numérico decimal, em que o dígito mais à direita tem um valor posicional de 1, e o próximo dígito à esquerda tem um valor posicional de 10, depois de 100, depois de 1.000, e assim por diante, no sistema binário, o dígito

mais à direita tem um valor posicional de 1, o próximo dígito à esquerda tem um valor posicional de 2, depois de 4, depois de 8 e assim por diante. Assim, o número 234 pode ser interpretado como  $4 * 1 + 3 * 10 + 2 * 100$ . O equivalente decimal do binário 1101 é  $1 * 1 + 0 * 2 + 1 * 4 + 1 * 8$  ou  $1 + 0 + 4 + 8$  ou 13.]

- 3.37 Qual é a velocidade do seu computador?** Como você pode determinar a velocidade com que seu computador realmente opera? Escreva um programa com um loop while que conte de 1 até 300.000.000 em intervalos de 1. Toda vez que o contador atingir um múltiplo de 100.000.000, mostre esse número na tela. Use seu relógio para marcar o tempo gasto entre cada repetição do loop por 100 milhões de vezes.

- 3.38** Escreva um programa que imprima 100 asteriscos, um de cada vez. Após cada décimo asterisco, seu programa deverá imprimir um caractere de nova linha. [Dica: conte de 1 até 100. Use o operador de módulo para reconhecer cada vez que o contador atingir um múltiplo de 10.]

- 3.39 Contando 7s.** Escreva um programa que leia um inteiro e determine e imprima quantos dígitos no inteiro são algarismos 7.

- 3.40 Padrão de asteriscos em xadrez.** Escreva um programa que apresente o seguinte padrão de tabuleiro de xadrez:

```
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Seu programa deverá usar apenas três instruções de saída, uma de cada das seguintes formas:

```
printf(" * ");
printf(" ");
printf("\n");
```

- 3.41 Múltiplos de 2 com um loop infinito.** Escreva um programa que continue imprimindo os múltiplos do inteiro 2, a saber 2, 4, 8, 16, 32, 64 e assim por diante. Seu loop não deverá terminar (ou seja, você deverá criar um loop infinito). O que acontece quando você executa esse programa?

- 3.42 Diâmetro, circunferência e área de um círculo.** Escreva um programa que leia o raio de um círculo (como um valor float) e calcule e imprima o diâmetro, a circunferência e a área. Use o valor 3.14159 para  $\pi$ .

- 3.43** O que há de errado com a instrução a seguir? Reescreva-a para obter o resultado que o programador provavelmente tentava obter.

```
printf("%d", ++(x + y));
```

- 3.44 Lados de um triângulo.** Escreva um programa que leia três valores `float` diferentes de zero, determine e imprima se eles poderiam representar os lados de um triângulo.
- 3.45 Lados de um triângulo retângulo.** Escreva um programa que leia três inteiros diferentes de zero e determine e imprima se eles poderiam ser os lados de um triângulo retângulo.
- 3.46 Fatorial.** O fatorial de um inteiro não negativo  $n$  é escrito como  $n!$  (pronuncia-se ‘ $n$  fatorial’) e é definido da seguinte forma:
- $$n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1 \quad (\text{para valores de } n \text{ maiores ou iguais a } 1)$$
- e
- $$n! = 1 \quad (\text{para } n = 0).$$

Por exemplo,  $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$ , que é 120.

- a)** Escreva um programa que leia um inteiro não negativo e calcule e imprima seu fatorial.
- b)** Escreva um programa que calcule o valor da constante matemática  $e$  usando a fórmula:

$$e = 1 + \frac{1}{1!} + \frac{1}{2!} + \frac{1}{3!} + \dots$$

- c)** Escreva um programa que calcule o valor de  $e^x$  usando a fórmula

$$e^x = 1 + \frac{x}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

## Fazendo a diferença

- 3.47 Calculadora de crescimento populacional mundial.** Use a Web para determinar a população mundial atual e sua taxa de crescimento anual. Escreva uma aplicação que leia esses valores, depois apresente a população mundial estimada após um, dois, três, quatro e cinco anos.
- 3.48 Calculadora da frequência cardíaca.** Ao se exercitar, você pode usar um monitor de batimentos cardíacos para ver se seus batimentos estão dentro de uma faixa segura sugerida por seus instrutores e médicos. De acordo com a American Heart Association (AHA) ([www.americanheart.org/presenter.jhtml?identifier=4736](http://www.americanheart.org/presenter.jhtml?identifier=4736)), a fórmula para calcular sua *frequência cardíaca máxima* em batimentos por minuto é 220 menos sua idade em anos. Sua *frequência cardíaca* está em uma faixa que é 50 a 85 por cento da máxima. [Nota: essas fórmulas são estimativas fornecidas pela AHA. As frequências cardíacas normal e máxima podem variar com base na saúde, condição física e sexo do indivíduo. Sempre consulte um médico ou profissional qualificado antes de iniciar ou modificar um programa de exercícios.] Crie um programa que leia a data de nascimento do usuário e o dia atual (consistindo cada um em dia, mês e ano). Seu programa deverá calcular e exibir a idade da pessoa (em anos), sua frequência cardíaca máxima e sua frequência cardíaca normal.
- 3.49 Impondo privacidade com criptografia.** O crescimento explosivo das comunicações pela Internet e do armazenamento de dados nos computadores conectados à Internet aumentou muito a preocupação com a privacidade. O campo da criptografia trata da codificação de dados para torná-los difíceis (ou impossíveis, com os esquemas mais avançados) de serem lidos por usuários não autorizados. Nesse exercício, você investigará um esquema simples de codificação e decodificação de dados. Uma empresa que queira enviar dados pela Internet lhe pediu que escrevesse um programa que os codificasse para que pudessem ser transmitidos com mais segurança. Todos os dados são transmitidos como inteiros de quatro dígitos. Sua aplicação deverá ler um inteiro de quatro dígitos informado pelo usuário e codificá-lo da seguinte forma: substitua cada dígito pelo resultado da soma de 7 ao dígito e calcular o módulo depois de dividir o novo valor por 10. Depois, troque o primeiro dígito pelo terceiro e troque o segundo dígito pelo quarto. A seguir, imprima o inteiro codificado. Escreva outra aplicação que receba um inteiro de quatro dígitos codificado e o decodifique (invertendo o esquema de codificação) para formar o número original. [Projeto de leitura opcional: pesquise ‘criptografia de chave pública’ em geral e o esquema de chave pública PGP (Pretty Good Privacy) específico. Você também poderá querer investigar o esquema RSA, que é bastante usado em aplicações de peso industrial.]